10. Errors Deeply: Wrap, Is, As

10. Errors Deeply: Wrap, Is, As

Phase: Phase 2 — Core language depth
Estimated time: 1–2 days
Slug: `10-errors-deep`

Learning objectives

- Wrap errors with context using fmt.Errorf and %w
- Use errors.Is and errors.As for inspection
- Choose sentinel vs custom error types
- Design error messages for operators and callers

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Wrapped errors preserve context for humans and machines. errors.Is/As let callers branch on meaning without string matching.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- `fmt.Errorf("...: %w", err)` wraps while keeping the chain.
- `errors.Is` finds sentinels in the chain; `errors.As` finds typed errors.
- Sentinel vars (ErrNotFound) are stable public contracts.
- Log at the edge; return errors from libraries without double-logging.
- Error strings should be lowercase and not punctuated like sentences (style).

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-errors && cd /tmp/gw-errors
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import (
    "errors"
    "fmt"
)

var ErrNotFound = errors.New("not found")

func load(id string) error {
    return fmt.Errorf("load %s: %w", id, ErrNotFound)
}

func main() {
    err := load("42")
    fmt.Println(err)
    fmt.Println(errors.Is(err, ErrNotFound))
}
```

Run it:

```
go run .
```

Expected output:

```
load 42: not found
true
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Wrap ErrNotFound and assert with errors.Is.
2. Define a typed ValidationError and retrieve it with errors.As.
3. Compare wrongly with err.Error()== and then fix to Is/As.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Wrapping

```go
if err != nil {
    return fmt.Errorf("load config %s: %w", path, err)
}
```

`%w` preserves the chain for `errors.Is` / `errors.As`.

Sentinel errors

```go
var ErrNotFound = errors.New("not found")
```

Good for stable, package-level conditions. Do not compare error strings.

Typed errors

```
type ValidationError struct{ Field string }
func (e *ValidationError) Error() string { return e.Field + " invalid" }

var ve *ValidationError
if errors.As(err, &ve) { /* ... */ }
```

Guideline

Return errors; log at the edge (main/handler). Avoid logging and returning the same error repeatedly.

Failure modes to watch while coding

- Comparing `err.Error() == "..."`
- Using `%v` instead of `%w` when wrapping
- Creating unique error strings instead of sentinels

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Python exception types vs Go error values + wrapping.
- Do not parse error strings — use Is/As.
- `raise from` ≈ `%w` wrapping, but Go keeps it a value.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

When wrapping, put the verb and resource first: `open config: %w` helps grepping logs.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Errors Deeply: Wrap, Is, As” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked

example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Comparing `err.Error() == "..."`
- Using `%v` instead of `%w` when wrapping
- Creating unique error strings instead of sentinels

Exercises

Work in `exercises/10-errors-deep/`. Read that folder's `README.md` before you code.

What you will build

Wrap errors with %w and detect sentinels via errors.Is in the exercise stubs.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/10-errors-deep/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Difference between errors.Is and errors.As?
2. When prefer sentinel vs typed error?
3. Where should you log errors?

Further reading

- https://go.dev/blog/go1.13-errors
- https://pkg.go.dev/errors